

Voz del Ciudadano

Plataforma de Iniciativas Legislativas Ciudadanas

Evidencias del Proyecto

Patrones Estructurales de Diseño de Software

Curso: Desarrollo de Software
Proyecto: PC3 — Sistema Voz del Ciudadano
Tecnología: Python 3.11 + Flask + Bootstrap 5
Repositorio: github.com/AndreSanchezVega/DS-PC3
Fecha: 5 de junio de 2026

Patrones Estructurales Implementados

#	Patrón	Propósito
1	Composite	Árbol jerárquico del documento
2	Proxy	Control de acceso a firma y creación
3	Decorator	Comentarios, recursos y modificaciones
4	Adapter	Verificación de identidad multi-proveedor
5	Facade	Orquesta todos los subsistemas

Índice

1. Repositorio GitHub	2
1.1. Página principal del repositorio	2
1.2. Pull Requests cerrados	3
1.3. Red de ramas (Network Graph)	4
1.4. Historial de commits en main	6
2. Demo de la Aplicación	7
2.1. Pantalla 1 — Dashboard	7
2.2. Pantalla 2 — Detalle de Propuesta y árbol Composite	9
2.3. Patrón Proxy — Firma exitosa	10
2.4. Patrón Proxy — Error de acceso denegado	11
3. Pruebas Automatizadas	12
3.1. Ejecución de pytest — 22 casos de prueba en verde	12
4. Código Fuente — Patrones Estructurales	13
4.1. Código fuente — Patrón Proxy (proxy.py)	13
4.2. Carpeta de patrones en GitHub	14

1. Repositorio GitHub

1.1. Página principal del repositorio

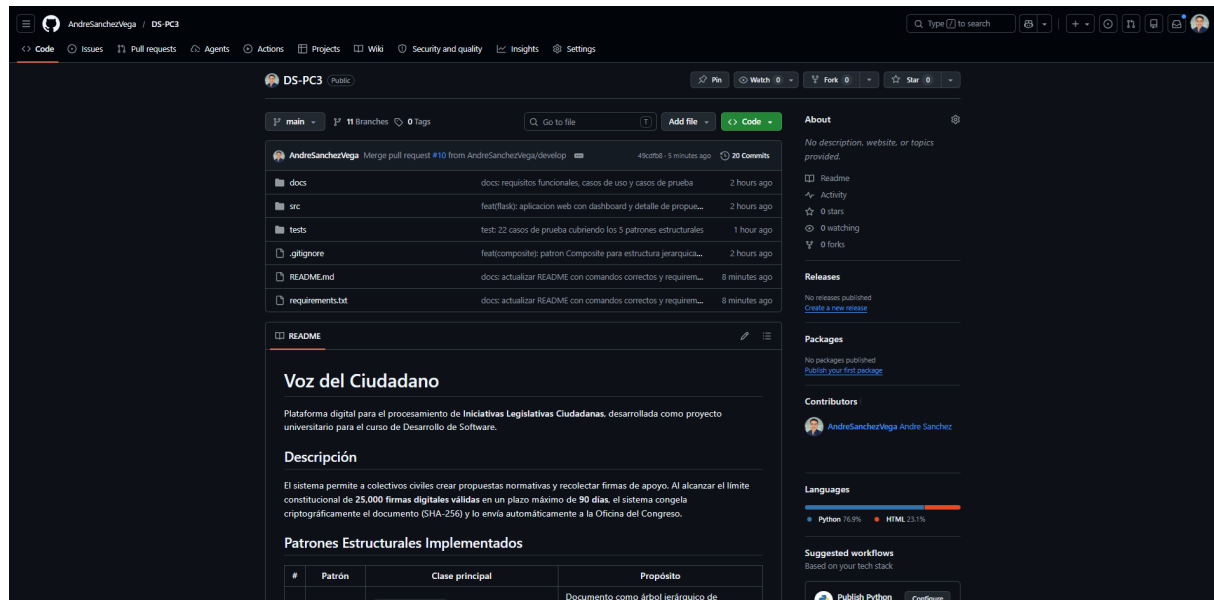


Figura 1: Repositorio DS-PC3 — Rama main en GitHub

Descripción: La imagen muestra la página principal del repositorio público **AndreSanchezVega/DS-PC3** alojado en GitHub. Se puede observar la rama activa **main** con un total de **20 commits** y **11 ramas** generadas durante el desarrollo. La estructura de carpetas refleja la arquitectura del proyecto: **docs/** contiene la documentación del sistema (requisitos funcionales, casos de uso y casos de prueba); **src/** agrupa los modelos de dominio, los cinco patrones estructurales y las plantillas HTML de la aplicación Flask; y **tests/** contiene la suite de pruebas automatizadas con pytest. Cada archivo muestra su último mensaje de commit asociado, evidenciando un flujo de trabajo profesional con commits descriptivos en formato *conventional commits*. En la sección derecha se visualiza la distribución de lenguajes del proyecto: **Python 76.9%** (lógica de negocio y patrones) y **HTML 23.1%** (plantillas Bootstrap 5). El **README.md** se renderiza automáticamente mostrando el nombre del sistema, su descripción y la tabla de patrones estructurales implementados.

1.2. Pull Requests cerrados

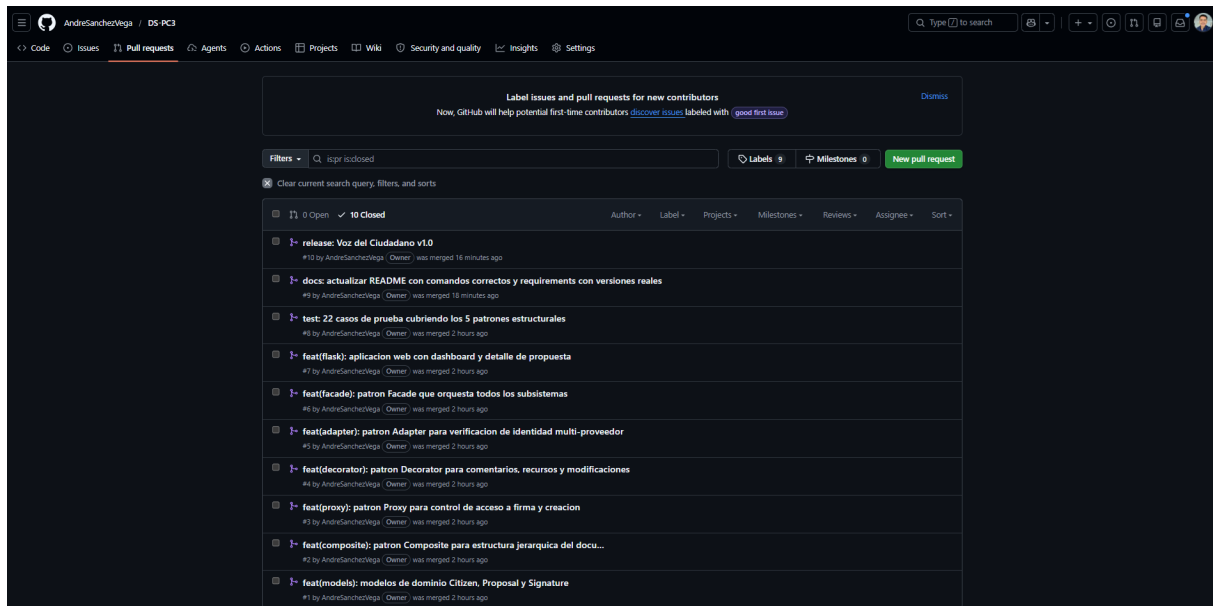


Figura 2: Historial completo de Pull Requests — 10 PRs mergeados

Descripción: La imagen muestra la pestaña *Pull Requests* del repositorio con el filtro `is:closed`, evidenciando un total de **10 Pull Requests cerrados y mergeados**, todos realizados por el propietario del repositorio **AndreSanchezVega**. El historial refleja el flujo de trabajo *Gitflow* adoptado durante el desarrollo, donde cada funcionalidad fue desarrollada en su propia rama `feature/` y posteriormente integrada a `develop` mediante un PR independiente antes de llegar a `main`.

El orden cronológico inverso de los PRs evidencia la secuencia de desarrollo seguida:

1. **PR #1** — `feat(models)`: modelos de dominio Citizen, Proposal y Signature.
2. **PR #2** — `feat(composite)`: patrón Composite para la estructura jerárquica del documento.
3. **PR #3** — `feat(proxy)`: patrón Proxy para control de acceso a firma y creación de propuestas.
4. **PR #4** — `feat(decorator)`: patrón Decorator para comentarios, recursos y modificaciones.
5. **PR #5** — `feat(adapter)`: patrón Adapter para verificación de identidad multi-proveedor.
6. **PR #6** — `feat(facade)`: patrón Facade que orquesta todos los subsistemas.
7. **PR #7** — `feat(flask)`: aplicación web con dashboard y detalle de propuesta.
8. **PR #8** — `test`: 22 casos de prueba cubriendo los 5 patrones estructurales.
9. **PR #9** — `docs`: actualización de README y `requirements.txt` con versiones reales.

10. **PR #10 — release:** Voz del Ciudadano v1.0, merge final de **develop** hacia **main**.

Cada PR tiene nombre descriptivo siguiendo la convención *conventional commits*, lo que permite trazar directamente qué patrón o funcionalidad fue integrada en cada iteración.

1.3. Red de ramas (Network Graph)

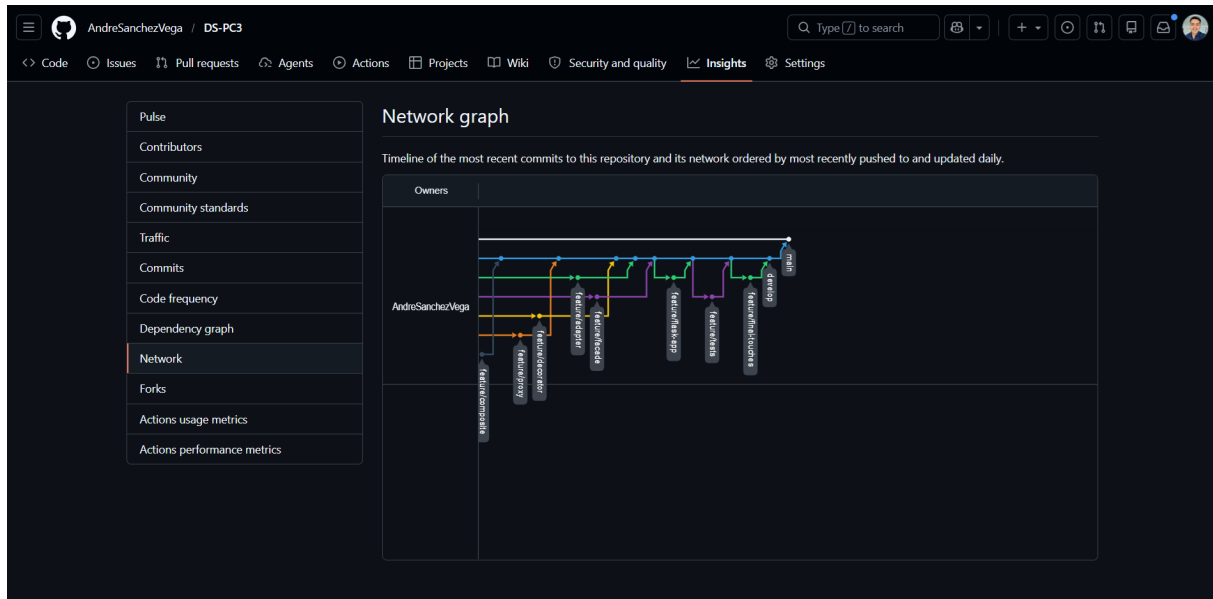


Figura 3: Network Graph — Árbol de ramas y commits del repositorio

Descripción: La imagen muestra el **Network Graph** del repositorio, accesible desde la pestaña *Insights* → *Network*. Este gráfico representa visualmente el historial completo de ramas y commits del proyecto, evidenciando la aplicación de la estrategia **Gitflow** durante todo el desarrollo.

Se identifican claramente las siguientes ramas, cada una representada con un color distinto:

- **main** (blanco) — rama principal de producción, ubicada en la parte superior. Recibe únicamente el merge final del release v1.0.
- **develop** (azul claro) — rama de integración continua donde confluyen todas las ramas **feature/** antes de llegar a **main**.
- **feature/composite** (gris) — implementación del patrón Composite.
- **feature/proxy** (naranja) — implementación del patrón Proxy.
- **feature/decorator** (naranja oscuro) — implementación del patrón Decorator.
- **feature/adaptor** (amarillo) — implementación del patrón Adapter.
- **feature/facade** (verde) — implementación del patrón Facade.

- `feature/flask-app` (teal) — aplicación web Flask con las dos pantallas del demo.
- `feature/tests` (morado) — suite completa de pruebas automatizadas con `pytest`.
- `feature/final-touches` (morado claro) — ajustes finales de documentación y `requirements.txt`.

El patrón de bifurcación y reintegración visible en el gráfico confirma que cada patrón estructural fue desarrollado de forma aislada en su propia rama, integrado a `develop` mediante Pull Request, y finalmente promovido a `main` en el release final. Esto refleja un flujo de trabajo profesional equivalente al utilizado en proyectos de software reales.

1.4. Historial de commits en main

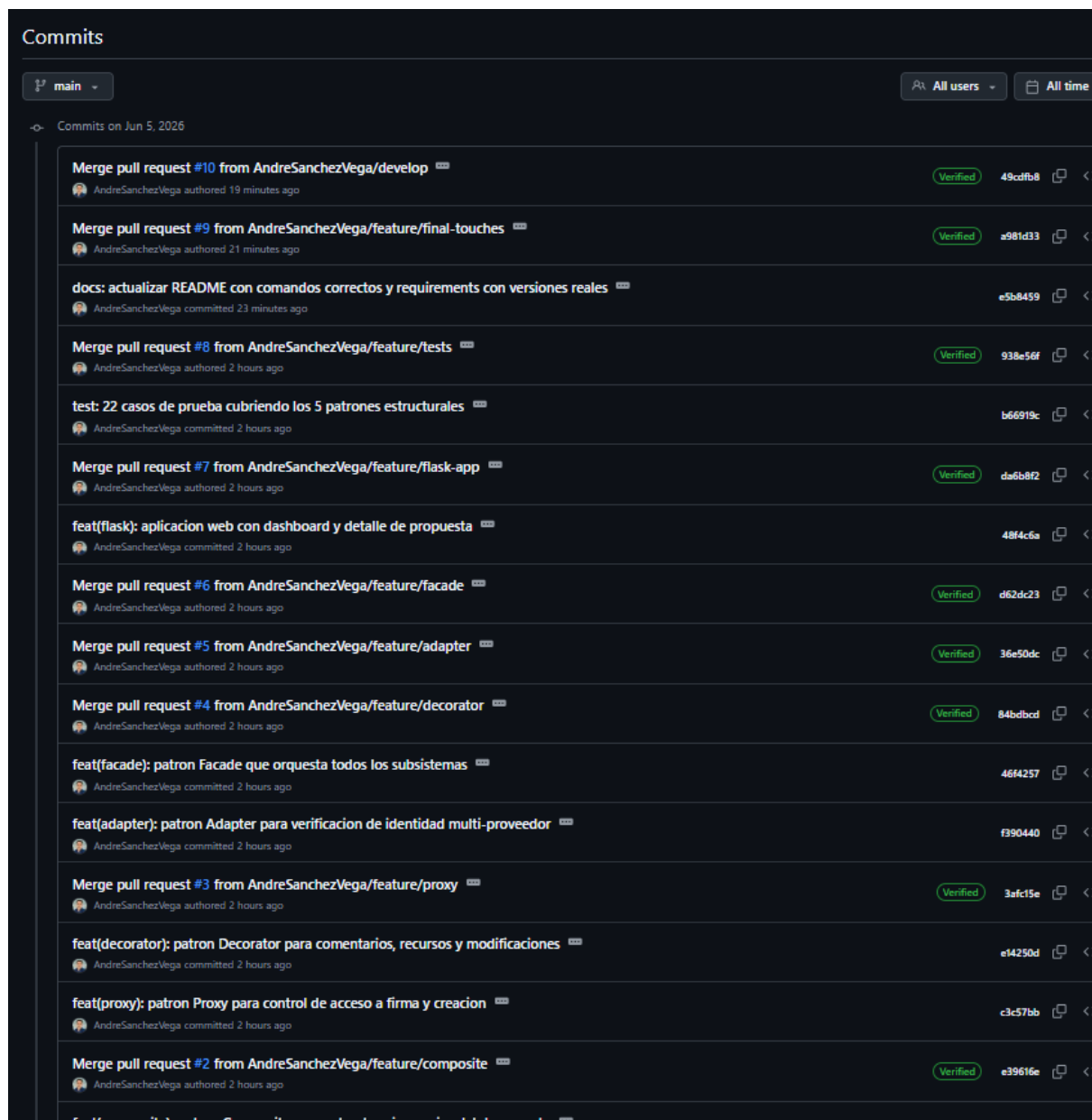


Figura 4: Historial de commits en la rama main — Jun 5, 2026

Descripción: La imagen muestra el historial completo de commits de la rama `main`, filtrado por *All users* y *All time*. Todos los commits fueron realizados el **5 de junio de 2026** por **AndreSanchezVega**, autor único del proyecto.

Se distinguen dos tipos de entradas en el historial:

- **Merge commits** (etiqueta verde *Verified*): son los commits generados automáticamente al cerrar un Pull Request. Aparecen con el formato `Merge pull request #N from AndreSanchezVega/feature/...` y confirman la integración de cada rama al flujo principal. Se identifican los merges de los PRs #2 al #10, correspondientes a cada patrón, la app Flask, los tests, los ajustes finales y el release v1.0.

- **Commits de feature:** son los commits directos de implementación, cada uno con un mensaje descriptivo en formato *conventional commits*:
 - **feat(composite):** patrón Composite para estructura jerárquica del documento.
 - **feat(proxy):** patrón Proxy para control de acceso a firma y creación.
 - **feat(decorator):** patrón Decorator para comentarios, recursos y modificaciones.
 - **feat(adapter):** patrón Adapter para verificación de identidad multi-proveedor.
 - **feat(facade):** patrón Facade que orquesta todos los subsistemas.
 - **feat(flask):** aplicación web con dashboard y detalle de propuesta.
 - **test:** 22 casos de prueba cubriendo los 5 patrones estructurales.
 - **docs:** actualización de README con comandos correctos y `requirements.txt` con versiones reales.

El historial demuestra una progresión ordenada e incremental del proyecto, donde cada commit representa una unidad lógica de trabajo independiente, facilitando la trazabilidad de cada decisión de diseño tomada durante el desarrollo.

2. Demo de la Aplicación

2.1. Pantalla 1 — Dashboard

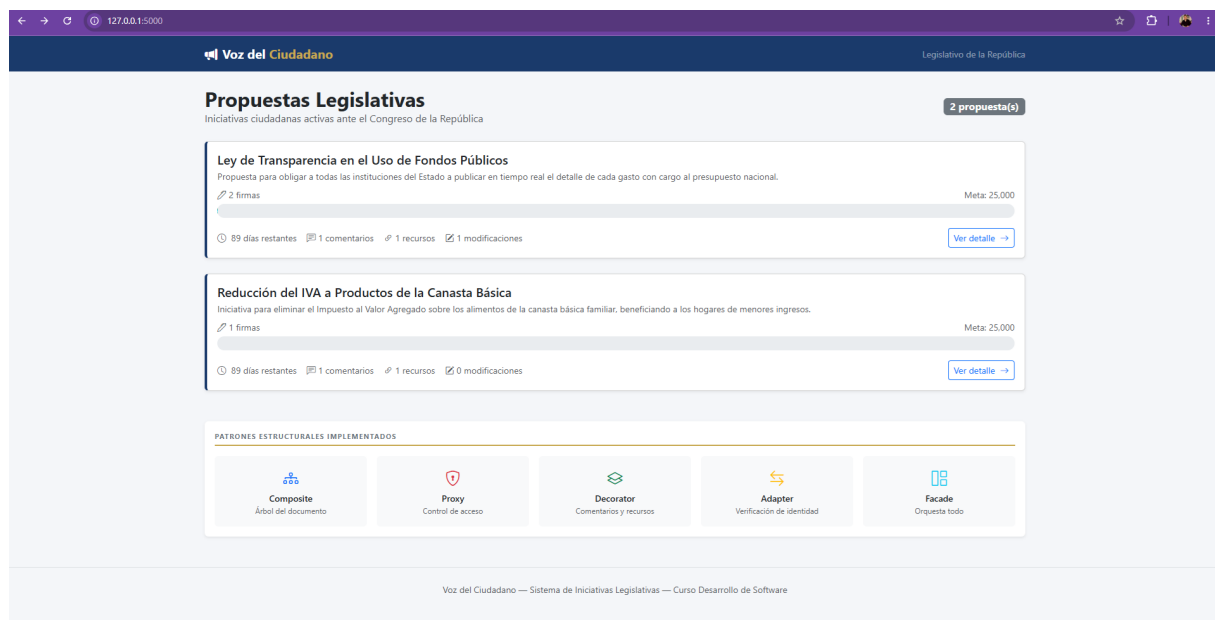


Figura 5: Pantalla 1 — Dashboard de Propuestas Legislativas (127.0.0.1:5000)

Descripción: La imagen muestra la **primera pantalla** de la aplicación web, accesible en la ruta raíz /. Corresponde al caso de uso **CU-02: Ver listado de propuestas**.

La interfaz está construida con **Flask + Bootstrap 5** y se compone de tres zonas principales:

- **Barra de navegación** (fondo azul institucional): muestra el nombre del sistema *Voz del Ciudadano* con el logo de megáfono y la leyenda *Legislativo de la República* en el extremo derecho.
- **Listado de propuestas:** se muestran las **2 propuestas** cargadas en los datos de demostración, cada una en una tarjeta con borde izquierdo azul que incluye:
 - Título y descripción breve de la iniciativa.
 - Contador de firmas con barra de progreso hacia la meta de **25,000 firmas** (propuesta #1 con 2 firmas, propuesta #2 con 1 firma).
 - Días restantes del plazo de 90 días (**89 días** en ambas propuestas).
 - Contadores de comentarios, recursos y modificaciones adjuntos, gestionados por el patrón **Decorator**.
 - Botón *Ver detalle* que dirige a la segunda pantalla.
- **Panel de patrones estructurales:** sección inferior que muestra los 5 patrones implementados en el sistema, cada uno con su ícono, nombre y descripción breve: **Composite** (árbol del documento), **Proxy** (control de acceso), **Decorator** (comentarios y recursos), **Adapter** (verificación de identidad) y **Facade** (orquestra todo).

Esta pantalla es gestionada internamente por **ProposalFacade** a través del método `get_all_proposals()`, que recupera y ordena todas las propuestas del sistema sin que la capa de presentación conozca los subsistemas subyacentes.

2.2. Pantalla 2 — Detalle de Propuesta y árbol Composite

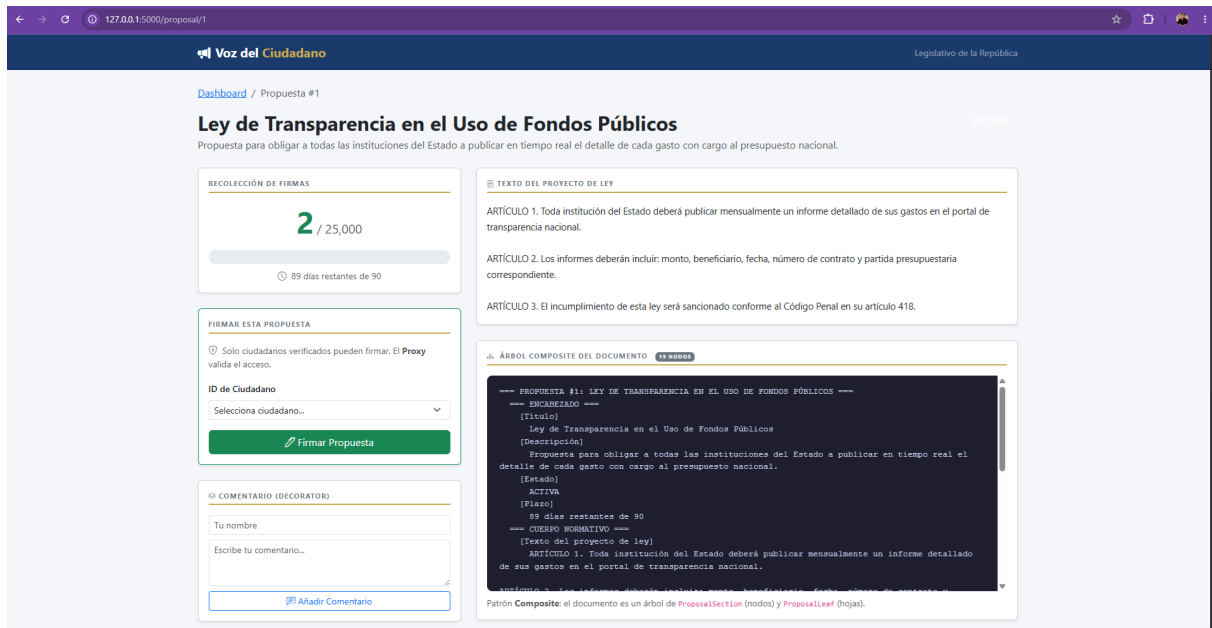


Figura 6: Pantalla 2 — Detalle de la Propuesta #1 con árbol Composite (127.0.0.1:5000/proposal/1)

Descripción: La imagen muestra la **segunda pantalla** de la aplicación, correspondiente al caso de uso **CU-03: Ver detalle de una propuesta**. Se accede desde el botón *Ver detalle* del dashboard y presenta toda la información de la propuesta **“Ley de Transparencia en el Uso de Fondos Públicos”**.

La pantalla se organiza en dos columnas:

Columna izquierda — Acciones e información:

- **Recolección de Firmas:** muestra el contador **2 / 25,000** con barra de progreso y los 89 días restantes del plazo constitucional de 90 días.
- **Firmar esta Propuesta** (patrón **Proxy**): formulario con selector de ciudadano y botón verde *Firmar Propuesta*. La nota indica explícitamente que *“Solo ciudadanos verificados pueden firmar. El Proxy valida el acceso”*, evidenciando el patrón `CitizenVerificationProxy` en funcionamiento.
- **Comentario (Decorator):** formulario para añadir comentarios mediante el patrón `CommentDecorator`, con campos de nombre y texto.

Columna derecha — Contenido del documento:

- **Texto del Proyecto de Ley:** muestra los tres artículos de la propuesta normativa con formato legible.
- **Árbol Composite del Documento** (patrón **Composite**): bloque de terminal oscuro que renderiza el árbol jerárquico completo del documento con **19 nodos**. Se visualizan las secciones **ENCABEZADO** (con hojas `[Título]`, `[Descripción]`, `[Estado]`,

[Plazo]) y CUERPO NORMATIVO (con el texto articulado). Al pie se indica: “*Patrón Composite: el documento es un árbol de ProposalSection (nodos) y ProposalLeaf (hojas)*”, conectando directamente la interfaz con el patrón de diseño implementado.

2.3. Patrón Proxy — Firma exitosa

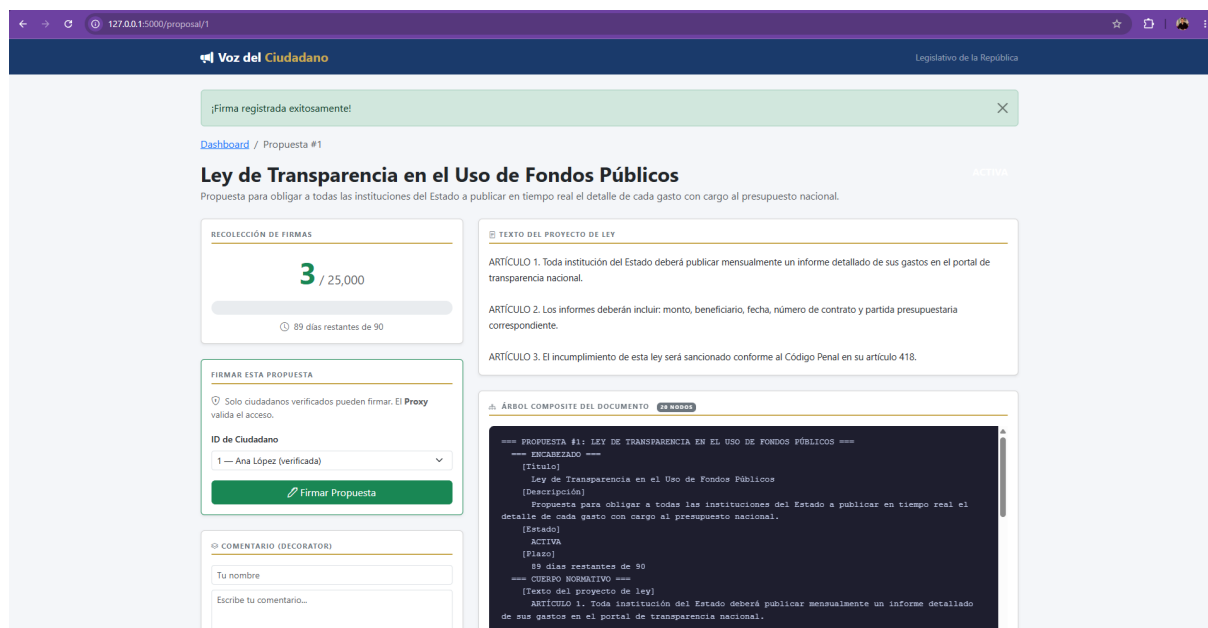


Figura 7: Patrón Proxy — Firma registrada exitosamente (127.0.0.1:5000/proposal/1)

Descripción: La imagen evidencia el funcionamiento del **patrón estructural Proxy** durante una operación de firma exitosa, correspondiente al caso de uso **CU-05: Firmar una propuesta** y al caso de prueba **CP-06**.

Se pueden observar los siguientes elementos que confirman la ejecución correcta del patrón:

- **Alerta verde** en la parte superior de la pantalla con el mensaje “*¡Firma registrada exitosamente!*”, generada por Flask tras la validación y registro exitoso de la firma.
- **Contador actualizado:** el marcador de firmas pasó de **2** a **3 / 25,000**, confirmando que la firma fue registrada correctamente en el sistema y que el contador de la propuesta se incrementó en tiempo real.
- **Barra de progreso:** refleja el avance actualizado hacia la meta de 25,000 firmas.
- **Ciudadano seleccionado:** el selector muestra “*1 — Ana López (verificada)*”, confirmando que el **CitizenVerificationProxy** permitió la operación al verificar que: (1) la ciudadana tiene estado **VERIFICADO**, (2) la propuesta está en estado **ACTIVA** y (3) Ana López no había firmado previamente esta propuesta.
- **Árbol Composite actualizado:** el bloque terminal ahora muestra **20** nodos

(antes tenía 19), reflejando que el nuevo registro de firma fue incorporado automáticamente al árbol jerárquico del documento mediante el patrón **Composite**.

El flujo completo fue coordinado por `ProposalFacade` a través de su método `sign_proposal()`, que delega al `CitizenVerificationProxy` antes de registrar la firma en el servicio real.

2.4. Patrón Proxy — Error de acceso denegado

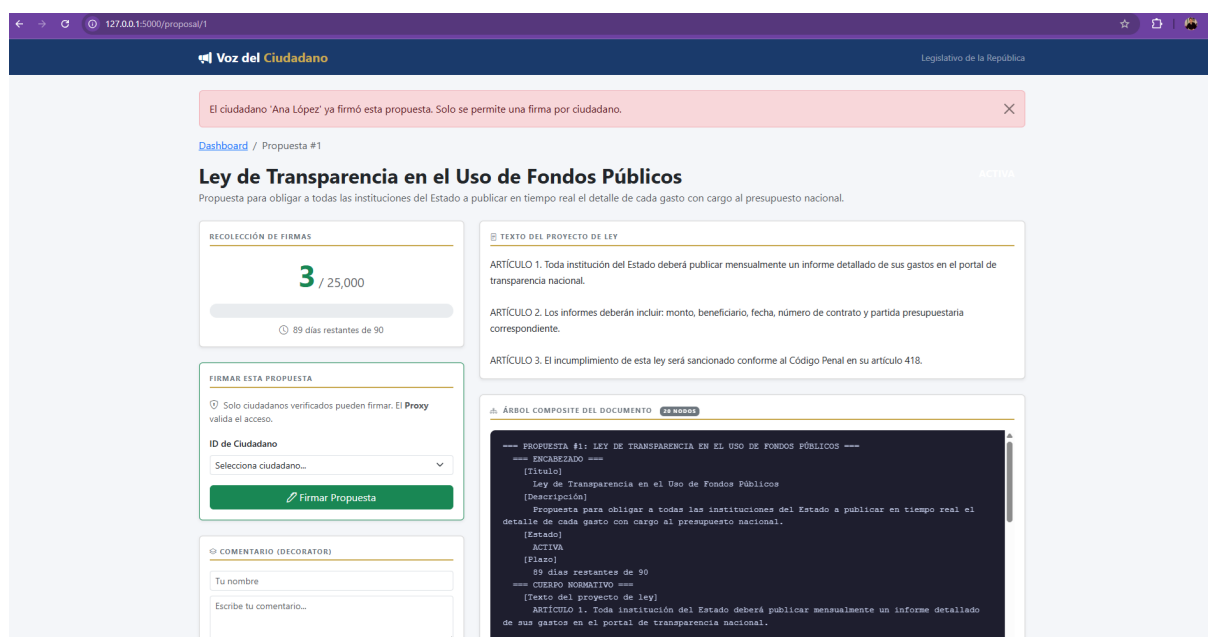


Figura 8: Patrón Proxy — Bloqueo de firma duplicada (127.0.0.1:5000/proposal/1)

Descripción: La imagen evidencia el funcionamiento del patrón estructural Proxy ante un intento de violación de la regla de unicidad de firma, correspondiente al caso de prueba **CP-07: Ciudadano intenta firmar dos veces la misma propuesta**.

Se observan los siguientes elementos:

- **Alerta roja** en la parte superior con el mensaje exacto: “El ciudadano 'Ana López' ya firmó esta propuesta. Solo se permite una firma por ciudadano.” Este mensaje es generado directamente por el método `_assert_not_already_signed()` del `CitizenVerificationProxy`, que lanza un `ValueError` al detectar que el `citizen_id` de Ana López ya está registrado en la lista de firmas de la propuesta.
- **Contador sin cambio:** el marcador permanece en **3 / 25,000**, confirmando que el Proxy bloqueó la operación antes de que llegara al servicio real `ProposalService`, por lo que no se registró ninguna firma adicional.
- **Integridad del árbol Composite:** el contador de nodos mantiene **20 nodos**, evidenciando que el documento no fue modificado por el intento fallido.

Este comportamiento demuestra la utilidad del patrón **Proxy** como capa de protección: el cliente (Flask) no necesita implementar ninguna validación de negocio, ya que el **CitizenVerificationProxy** actúa como guardián aplicando las tres reglas en secuencia antes de delegar al objeto real: (1) ciudadano verificado, (2) propuesta activa, (3) firma no duplicada. Al fallar cualquiera de ellas, la operación es rechazada con un mensaje descriptivo sin afectar el estado del sistema.

3. Pruebas Automatizadas

3.1. Ejecución de pytest — 22 casos de prueba en verde

```

D:\DS-PC3>py -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Andre\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
rootdir: D:\DS-PC3
collected 22 items

tests/test_patterns.py::TestCiudadanos::test_CP01_registro_exitoso PASSED [ 4%]
tests/test_patterns.py::TestCiudadanos::test_CP02_registro_falla_correo_duplicado PASSED [ 9%]
tests/test_patterns.py::TestCiudadanos::test_CP03_verificacion_adapter_exitosa PASSED [ 13%]
tests/test_patterns.py::TestCiudadanos::test_CP04_verificacion_adapter_falla_dpi_invalido PASSED [ 18%]
tests/test_patterns.py::TestProxy::test_CP05_proxy_bloquea_ciudadano_no_verificado PASSED [ 22%]
tests/test_patterns.py::TestProxy::test_CP06_proxy_permite_firma_exitosa PASSED [ 27%]
tests/test_patterns.py::TestProxy::test_CP07_proxy_bloquea_firma_duplicada PASSED [ 31%]
tests/test_patterns.py::TestProxy::test_CP08_proxy_bloquea_propuesta_congelada PASSED [ 36%]
tests/test_patterns.py::TestProxy::test_CP18_proxy_bloquea_crear_propuesta_sin_verificar PASSED [ 40%]
tests/test_patterns.py::TestComposite::test_CP09_renderizado_arbol_composite PASSED [ 45%]
tests/test_patterns.py::TestComposite::test_CP10_conteo_nodos_composite PASSED [ 50%]
tests/test_patterns.py::TestComposite::test_composite_hoja_es_nodo_simple PASSED [ 54%]
tests/test_patterns.py::TestComposite::test_composite_seccion_acumula_hijos PASSED [ 59%]
tests/test_patterns.py::TestDecorator::test_CP11_añadir_comentario PASSED [ 63%]
tests/test_patterns.py::TestDecorator::test_CP12_añadir_recurso PASSED [ 68%]
tests/test_patterns.py::TestDecorator::test_CP13_añadir_modificacion PASSED [ 72%]
tests/test_patterns.py::TestDecorator::test_CP14_encadenamiento_multiples_decorators PASSED [ 77%]
tests/test_patterns.py::TestDecorator::test_decorator_rechaza_url_invalida PASSED [ 81%]
tests/test_patterns.py::TestDecorator::test_decorator_rechaza_modificacion_no_proponente PASSED [ 86%]
tests/test_patterns.py::TestFacade::test_CP15_crear_propuesta_via_facade PASSED [ 90%]
tests/test_patterns.py::TestFacade::test_CP16_congelamiento_automatiko_al_alcanzar_25000 PASSED [ 95%]
tests/test_patterns.py::TestFacade::test_CP17_envio_al_congreso_tras_congelamiento PASSED [100%]

===== 22 passed in 0.48s =====

```

Figura 9: Ejecución de pytest — 22/22 casos de prueba aprobados (D:\DS-PC3>py -m pytest tests/ -v)

Descripción: La imagen muestra la ejecución completa de la suite de pruebas automatizadas desde el **Símbolo del sistema** de Windows, ejecutada en el directorio raíz D:\DS-PC3 con el comando `py -m pytest tests/ -v`.

El encabezado de la sesión indica: **Python 3.11.0, pytest-9.0.3, pluggy-1.6.0**, con **22 items recolectados** del archivo `tests/test_patterns.py`.

Los 22 casos están organizados en 5 clases de prueba, cada una cubriendo un patrón estructural distinto:

- **TestCiudadanos** (CP-01 a CP-04): registro exitoso, correo duplicado, verificación exitosa con el patrón **Adapter** y rechazo de DPI inválido.
- **TestProxy** (CP-05 a CP-08, CP-18): bloqueo de ciudadano no verificado, firma exitosa, bloqueo de firma duplicada, bloqueo sobre propuesta congelada y bloqueo de creación sin verificar.

- **TestComposite** (CP-09, CP-10 + 2 adicionales): renderizado del árbol, conteo de nodos, verificación de hoja como nodo simple y acumulación recursiva de hijos en secciones.
- **TestDecorator** (CP-11 a CP-14 + 2 adicionales): adición de comentario, recurso y modificación, encadenamiento de múltiples decorators, rechazo de URL inválida y bloqueo de modificación por no proponente.
- **TestFacade** (CP-15 a CP-17): creación de propuesta vía Facade, congelamiento automático SHA-256 al alcanzar exactamente 25,000 firmas, y registro de envío al Congreso con hash de 64 caracteres.

La línea final confirma el resultado: **22 passed in 0.48s**, con todos los nombres de test resaltados en **verde**, indicando que la totalidad de los casos de prueba definidos en `docs/casos_de_prueba.md` fueron implementados y ejecutados satisfactoriamente.

4. Código Fuente — Patrones Estructurales

4.1. Código fuente — Patrón Proxy (`proxy.py`)

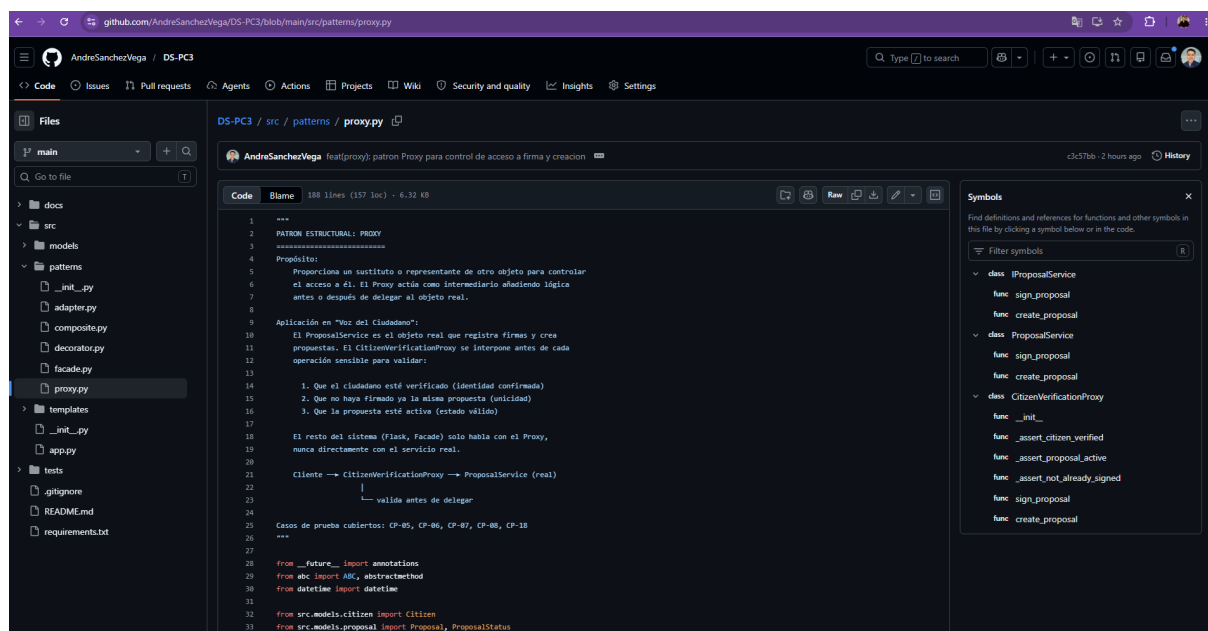


Figura 10: Código fuente del patrón Proxy en GitHub — `src/patterns/proxy.py`

Descripción: La imagen muestra el archivo `proxy.py` visualizado directamente en GitHub, rama `main`, con **188 líneas** (157 líneas de código, 6.32 KB). El commit asociado es `c3c57bb`: *“feat(proxy): patron Proxy para control de acceso a firma y creacion”*.

Se distinguen dos zonas principales:

Panel izquierdo — Código fuente: muestra el docstring completo del módulo (líneas 1–26) que documenta el patrón de forma exhaustiva:

- **Propósito** (líneas 4–7): define el patrón Proxy como sustituto que controla el acceso al objeto real, actuando como intermediario.
- **Aplicación en el sistema** (líneas 9–23): describe cómo `CitizenVerificationProxy` se interpone antes de cada operación sensible validando tres reglas: (1) ciudadano verificado, (2) firma no duplicada, (3) propuesta activa. Se muestra el diagrama textual del flujo: `Cliente → CitizenVerificationProxy → ProposalService`.
- **Casos de prueba cubiertos** (línea 25): referencia explícita a CP-05, CP-06, CP-07, CP-08 y CP-18.
- **Imports** (líneas 28–33): importa las clases `Citizen`, `Proposal` y `ProposalStatus` de la capa de modelos de dominio.

Panel derecho — Symbols: GitHub detecta y lista automáticamente los símbolos del archivo, evidenciando la estructura del patrón:

- `class IProposalService`: interfaz abstracta con métodos `sign_proposal` y `create_proposal`.
- `class ProposalService`: objeto real con la misma interfaz.
- `class CitizenVerificationProxy`: el proxy con sus métodos de validación privados (`_assert_citizen_verified`, `_assert_proposal_active`, `_assert_not_already_signed`) y los métodos públicos que delegan al servicio real.

Esta estructura de tres clases es la implementación canónica del patrón **Proxy** según GoF: interfaz común, objeto real y proxy que controla el acceso.

4.2. Carpeta de patrones en GitHub

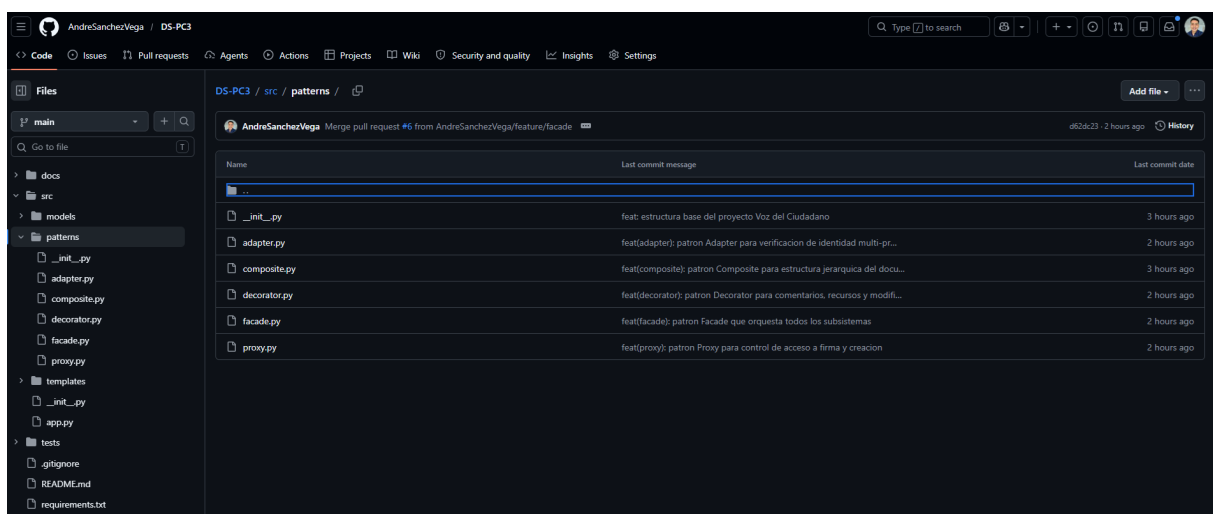


Figura 11: Carpeta `src/patterns/` en GitHub — 5 patrones estructurales

Descripción: La imagen muestra la carpeta `src/patterns/` del repositorio navegada desde GitHub, rama `main`. En el panel izquierdo se visualiza el árbol completo del proyecto con todas sus carpetas: `docs/`, `src/` (con subcarpetas `models/`, `patterns/` y `templates/`), y `tests/`, confirmando la estructura de capas del sistema.

En el panel central se listan los **5 archivos de patrones estructurales** implementados, cada uno con su mensaje de commit asociado que referencia explícitamente el patrón correspondiente:

- `adapter.py` — *“feat(adapter): patron Adapter para verificacion de identidad multi-pr...”*: implementa `IDVerificationAdapter`, `RegistroCivilAdapter` y `BiometricoAdapter` para adaptar servicios externos de verificación de identidad a una interfaz unificada `IIdentityVerifier`.
- `composite.py` — *“feat(composite): patron Composite para estructura jerarquica del docu...”*: implementa `ProposalComponent`, `ProposalLeaf`, `ProposalSection` y `ProposalDocument` para representar el documento de la propuesta como árbol jerárquico.
- `decorator.py` — *“feat(decorator): patron Decorator para comentarios, recursos y modifi...”*: implementa `CommentDecorator`, `ResourceDecorator` y `ModificationDecorator` para añadir dinámicamente elementos a una propuesta.
- `facade.py` — *“feat(facade): patron Facade que orchestra todos los subsistemas”*: implementa `ProposalFacade` como punto de entrada único que coordina los cuatro patrones anteriores.
- `proxy.py` — *“feat(proxy): patron Proxy para control de acceso a firma y creacion”*: implementa `CitizenVerificationProxy` e `IProposalService` para controlar el acceso a las operaciones sensibles del sistema.

Cada archivo fue introducido mediante un Pull Request independiente desde su rama `feature/` correspondiente, como se evidencia en el mensaje del último commit de la carpeta: *“Merge pull request #6 from AndreSanchezVega/feature/facade”*.